EECE 7205 Fundamentals of Computer Engineering
Binary Trees In-Class Work

Karishma Premnath

Fall 2026

Array used throughout: $arr = \{5, 12, 8, 21, 3, 17, 9\}$, $n = 7$.

PROBLEM 1

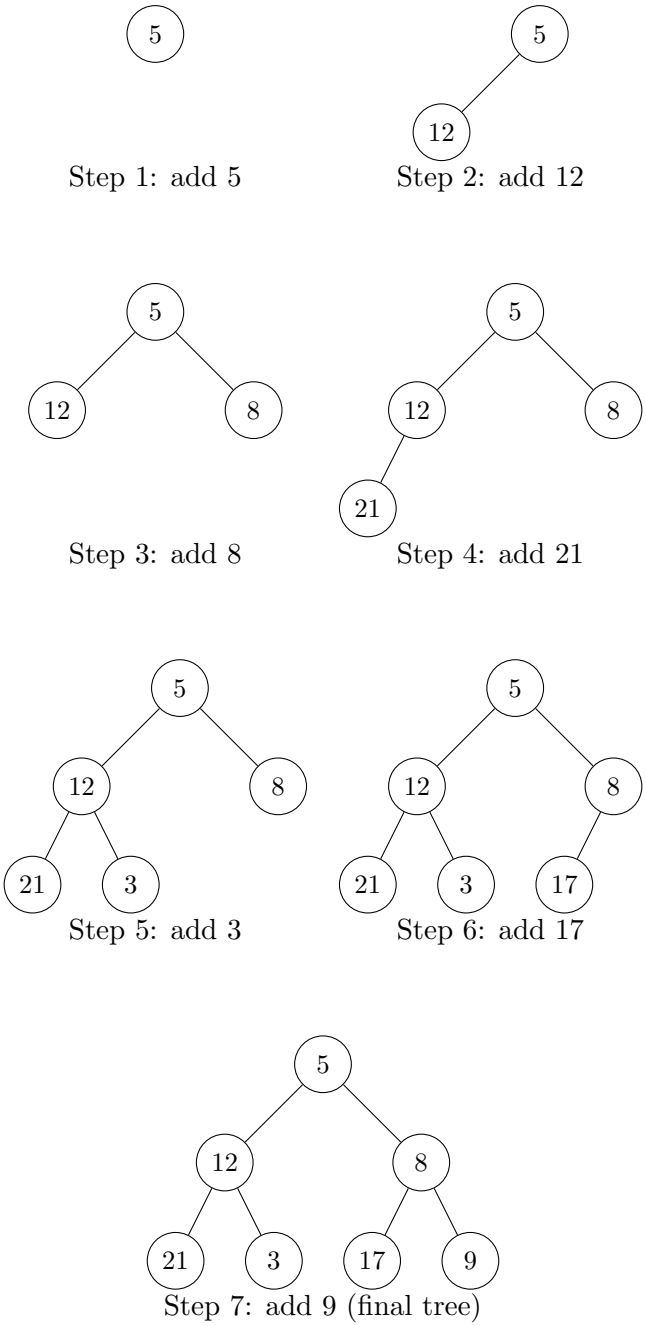
(a) Building the tree one element at a time

Step	Element	Index i	Parent index	Parent value	Left / Right	Rule(s)
1	5	0	none	none	root	C1
2	12	1	$(1-1)/2 = 0$	5	Left	C2, C3
3	8	2	$(2-1)/2 = 0$	5	Right	C2, C4
4	21	3	$(3-1)/2 = 1$	12	Left	C2, C3
5	3	4	$(4-1)/2 = 1$	12	Right	C2, C4
6	17	5	$(5-1)/2 = 2$	8	Left	C2, C3
7	9	6	$(6-1)/2 = 2$	8	Right	C2, C4

The side depends on whether the index is odd or even. Steps 2, 4 and 6 have odd indices 1, 3 and 5, so C3 sends them left. Steps 3, 5 and 7 have even indices 2, 4 and 6, so C4 sends them right.

Integer division is what lets two elements share a parent. Both $(1-1)/2$ and $(2-1)/2$ come out as 0 after truncation, so 12 and 8 both land under 5. The same thing happens at 3/2 and 4/2, which is how 21 and 3 end up under 12.

Tree after each step



(b) Nodes with NULL children

The nodes with NULL children are 21, 3, 17 and 9, sitting at indices 3, 4, 5 and 6. The rule that tells me this is C5.

To see why, I work out where each of their children would have to be:

Node	Index i	Child indices $2i+1$, $2i+2$
21	3	7, 8
3	4	9, 10
17	5	11, 12
9	6	13, 14

The largest index the array actually has is 6, because $n = 7$. Every index in that table is 7 or bigger, so there is no element to put in any of those slots. C5 covers exactly this case: if a child index reaches n, that child is NULL. Both pointers are NULL on all four nodes, which makes all four of them leaves.

(c) Parts of the final tree

- **Root:** 5. It is the element at index 0 and the only node with no parent above it.
- **Leaves:** 21, 3, 17 and 9. These are the four nodes from part (b) with no children.
- **Internal nodes:** 5, 12 and 8. I get these by taking the total minus the leaves, so $7 - 4 = 3$.
- **Edges:** 6, from $n - 1 = 7 - 1$. They are 5 to 12, 5 to 8, 12 to 21, 12 to 3, 8 to 17 and 8 to 9.
- **Height:** 2. The longest way down from the root to a leaf is 5 to 12 to 21, and that crosses 2 edges.
- **Depth of 17:** 2. Coming down from the root I pass 5, then 8, then 17, so 2 edges.
- **Ancestors of 3:** 12 and 5. Starting at index 4 the parent is $(4-1)/2 = 1$, which is 12, and the parent of index 1 is $(1-1)/2 = 0$, which is 5. A node is not counted as its own ancestor.
- **Descendants of 12:** 21 and 3. Index 1 has children at indices 3 and 4, and neither of those has children of its own, so the list stops there.
- **Subtree rooted at 8:** 8, 17 and 9, which is 3 nodes. A subtree is the node plus everything hanging below it.
- **Path from 5 to 9:** 5, 8, 9. That is 3 nodes, but the length is 2, because length counts edges rather than nodes.

(d) Checking the properties

Property	Formula	Value	Actual
Max nodes on level 2	$2^L = 2^2$	4	4 (21, 3, 17, 9)
Max nodes for height $H = 2$	$2^{H+1} - 1 = 2^3 - 1$	7	$n = 7$
Leaves	(two-child nodes) + $1 = 3 + 1$	4	4 leaves
Min height for $N = 7$	$\lfloor \log_2 7 \rfloor$	2	height 2

The most nodes level 2 can hold is $2^L = 2^2 = 4$. Level 2 really does hold 21, 3, 17 and 9, so it has 4 and the level is packed.

The most nodes a tree of height 2 can hold is $2^{H+1} - 1 = 2^3 - 1 = 7$. My tree has $n = 7$, so it is carrying as many nodes as a height of 2 allows.

The rule that leaves equal the number of two-child nodes plus one also holds. The nodes with two children are 5, 12 and 8, which is 3, and $3 + 1 = 4$. I counted 4 leaves back in part (b), so the two agree.

The smallest height 7 nodes can have is $\lfloor \log_2 7 \rfloor$. Since $\log_2 7$ is about 2.807, the floor is 2, and my tree has height 2 as well. So this is the shortest arrangement 7 nodes can take.

Is it full, complete, or both?

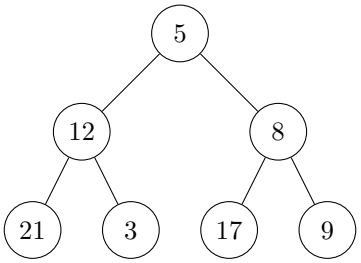
It is both.

The tree is complete because of C6. Every element goes under the parent at $(i - 1)/2$ on the side its index parity picks, so indices 0 through 6 get used in order and each level fills left to right before the next one starts. There is no way to skip a spot, so no gaps.

It is also full, since 5, 12 and 8 have two children each and 21, 3, 17 and 9 have none, so nobody has just one child. C6 does not promise this though. It only worked out because $n = 7$ is exactly $2^{H+1} - 1$ and the bottom level filled up. With 6 elements, 17 would sit alone at index 5 and the tree would still be complete but not full.

PROBLEM 2

All four traversals run on the final tree from Problem 1:



(a) Preorder (Node, Left, Right)

#	Visit	Why it is visited now	Rule	Output so far
1	5	Visit 5 first, then go into its left subtree at 12, then its right subtree at 8	T2	5
2	12	Visit 12, then its left subtree at 21, then its right one at 3	T2	5 12
3	21	Visit 21. Both of its subtrees are NULL so both calls come straight back	T2, T1, T6	5 12 21
4	3	Visit 3. Both subtrees are NULL here as well	T2, T1, T6	5 12 21 3
5	8	The left subtree of 5 is finished, so move into the right one and visit 8	T2	5 12 21 3 8
6	17	Visit 17, which is a leaf, so there is nothing under it	T2, T1, T6	5 12 21 3 8 17
7	9	Visit 9, also a leaf	T2, T1, T6	5 12 21 3 8 17 9

Preorder result: 5 12 21 3 8 17 9.

(b) Inorder (Left, Node, Right)

#	Visit	Why it is visited now	Rule	Output so far
1	21	Go left from 5 to 12 to 21. The left of 21 is NULL and returns, so 21 comes out first	T3, T1, T6	21
2	12	The left subtree of 12 is finished, so 12 is next	T3	21 12
3	3	Now the right subtree of 12. The left of 3 is NULL, so visit 3	T3, T1, T6	21 12 3
4	5	The whole left subtree of 5 is finished, so the root is visited	T3	21 12 3 5
5	17	Into the right subtree of 5. The left of 17 is NULL, so visit 17	T3, T1, T6	21 12 3 5 17
6	8	The left subtree of 8 is finished, so visit 8	T3	21 12 3 5 17 8
7	9	The left of 9 is NULL, so visit 9	T3, T1, T6	21 12 3 5 17 8 9

Inorder result: 21 12 3 5 17 8 9. The root comes out fourth of seven. Each subtree has 3 nodes, so 5 sits in the middle.

(c) Postorder (Left, Right, Node)

#	Visit	Why it is visited now	Rule	Output so far
1	21	21 is a leaf, so both of its subtree calls return at once and it is visited	T4, T1, T6	21
2	3	3 is a leaf as well	T4, T1, T6	21 3
3	12	Both children of 12 are finished, so 12 can be visited	T4	21 3 12
4	17	17 is a leaf	T4, T1, T6	21 3 12 17
5	9	9 is a leaf	T4, T1, T6	21 3 12 17 9
6	8	Both children of 8 are finished, so 8 can be visited	T4	21 3 12 17 9 8
7	5	Both subtrees of the root are finished, so 5 is visited last	T4	21 3 12 17 9 8 5

Postorder result: 21 3 12 17 9 8 5.

(d) Level order using a queue

Rule T5 is applied at every row. The queue starts holding only the root.

#	Queue before	Remove and visit	Add children (L, R)	Queue after	Output so far
1	5	5	12, 8	12, 8	5
2	12, 8	12	21, 3	8, 21, 3	5 12
3	8, 21, 3	8	17, 9	21, 3, 17, 9	5 12 8
4	21, 3, 17, 9	21	none, both NULL	3, 17, 9	5 12 8 21
5	3, 17, 9	3	none, both NULL	17, 9	5 12 8 21 3
6	17, 9	17	none, both NULL	9	5 12 8 21 3 17
7	9	9	none, both NULL	empty, stop	5 12 8 21 3 17 9

Level order result: 5 12 8 21 3 17 9.

Summary of the four traversals

Traversal	Rule	Order	Result
Preorder	T2	N L R	5 12 21 3 8 17 9
Inorder	T3	L N R	21 12 3 5 17 8 9
Postorder	T4	L R N	21 3 12 17 9 8 5
Level order	T5	level by level	5 12 8 21 3 17 9

(e) Why level order matches the array, and which traversal ends on the root

The level order output matches the array because of the way I built the tree. Creation took $arr[0]$ up to $arr[6]$ in order and put each one at tree position i, which is rule C6. The queue in T5 is first in, first out, so it pulls the nodes back out in that same order 0 to 6. So level order gives back the array. This is only true because of how I built this tree. It does not hold for binary trees in general.

Postorder visits the root last. T4 is L R N, so a node has to wait until both of its subtrees are done. The root's two subtrees hold the other six nodes, so the root cannot come out until all of them have. Preorder does the opposite. T2 visits the node first, so the root comes out first.

(f) Searching for 3 and for 4

Searching with level order means comparing against 5, 12, 8, 21, 3, 17, 9 in that order.

Looking for 3, I compare 5, then 12, then 8, then 21, then 3, which matches. That is **5 comparisons** and the search succeeds.

Looking for 4, I compare 5, 12, 8, 21, 3, 17 and 9, and none of them match. That is **7 comparisons** and the search fails.

This tree is not sorted, so a comparison does not tell me which way to go next. I have to check every node until the key turns up. The worst case is all $n = 7$ nodes, and a failed search always costs the full 7.

PROBLEM 3

Every insertion starts from where the previous one left off, so the tree grows from $n = 7$ up to $n = 11$ across parts (b) to (e). Shortcut I6 says the new element always goes to index n, and its parent is at $(n - 1)/2$. I use I6 to predict each answer first, then check it with the full level order scan.

(a) Inserting into an empty tree

The tree is empty, so there is no node to scan and no parent to attach to. Rule I1 covers this. The new node becomes the root, so 5 is the root and n goes from 0 to 1. This is the same as step 1 of Problem 1.

(b) Insert 11

The tree has $n = 7$. By I6 the new index is 7, the parent index is $(7 - 1)/2 = 3$ which holds 21, and 7 is odd so it is a left child. The scan below works through the nodes in level order using a queue, which is I2.

#	Node	Left, Right	Decision	Rule
1	5	12, 8	Both children are there, so put them in the queue and move on	I5
2	12	21, 3	Both children are there, so queue them and move on	I5
3	8	17, 9	Both children are there, so queue them and move on	I5
4	21	NULL, NULL	The left is free, so 11 goes here and the search stops	I3

So 11 becomes the left child of 21, which is what I6 predicted. Level order after: 5 12 8 21 3 17 9 11.

(c) Insert 2

The tree now has $n = 8$. By I6 the new index is 8, the parent index is $(8 - 1)/2 = 3$ which is 21 again, and 8 is even so it is a right child.

#	Node	Left, Right	Decision	Rule
1	5	12, 8	Both children are there, so queue them and move on	I5
2	12	21, 3	Both children are there, so queue them and move on	I5
3	8	17, 9	Both children are there, so queue them and move on	I5
4	21	11, NULL	The left is taken but the right is free, so 2 goes on the right	I4

So 2 ends up on the other side of the same parent. The reason is that 11 took the left slot in part (b). I3 is checked before I4, so when the left is full I3 fails and I4 is the rule that decides. The index arithmetic says the same thing. Indices 7 and 8 both have parent 3, and whether the index is odd or even picks the side. Level order after: 5 12 8 21 3 17 9 11 2.

(d) Insert 14

The tree has $n = 9$. By I6 the new index is 9, the parent index is $(9-1)/2 = 4$ which holds 3, and 9 is odd so it is a left child.

#	Node	Left, Right	Decision	Rule
1	5	12, 8	Both children are there, so skip it	I5
2	12	21, 3	Both children are there, so skip it	I5
3	8	17, 9	Both children are there, so skip it	I5
4	21	11, 2	Full now as well, so skip it	I5
5	3	NULL, NULL	The left is free, so 14 goes here	I3

The nodes skipped by I5 are 5, 12, 8 and 21. That is four of them, one more than in parts (b) and (c), because 21 filled up in part (c). So 14 becomes the left child of 3. Level order after: 5 12 8 21 3 17 9 11 2 14.

(e) Predicting where 6 goes with I6

Using I6 on its own, with no scanning at all. The tree has $n = 10$, so:

- new index $= n = 10$
- parent index $= (10-1)/2 = 9/2 = 4$ after integer division
- parent value = the node at index 4, which is 3
- 10 is even, so $10 = 2(4) + 2$ and it is the right child

Now the check:

#	Node	Left, Right	Decision	Rule
1	5	12, 8	Both children are there, so skip it	I5
2	12	21, 3	Both children are there, so skip it	I5
3	8	17, 9	Both children are there, so skip it	I5
4	21	11, 2	Both children are there, so skip it	I5
5	3	14, NULL	The left is taken, the right is free, so 6 goes on the right	I4

The scan agrees with the prediction, so 6 is the right child of 3. Level order after: 5 12 8 21 3 17 9 11 2 14 6.

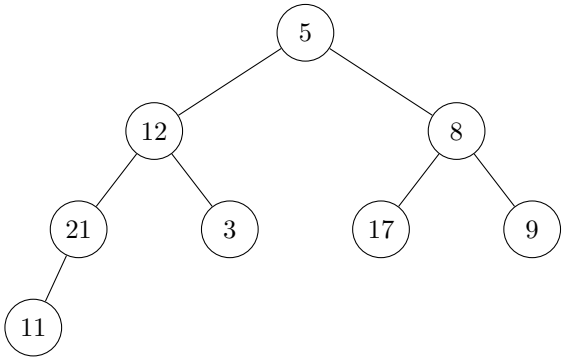
The scan had to look at 5 nodes to work this out. I6 got the same answer with one division. That is the point of keeping the tree complete. The next free spot is always index n, so finding it is arithmetic instead of a walk through the tree.

Insertion summary

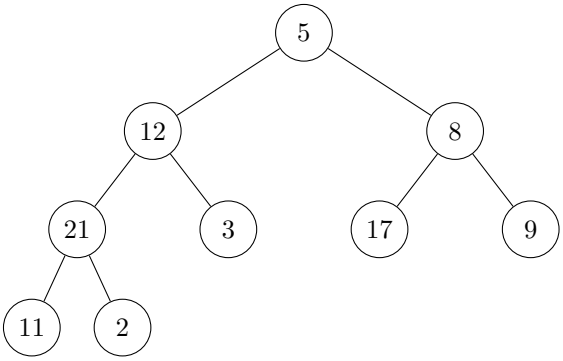
Insert	Nodes checked	Rule at each node	Parent	Left / Right	Level order after
11	5, 12, 8, 21	I5, I5, I5, I3	21 (index 3)	Left	5 12 8 21 3 17 9 11
2	5, 12, 8, 21	I5, I5, I5, I4	21 (index 3)	Right	5 12 8 21 3 17 9 11 2
14	5, 12, 8, 21, 3	I5, I5, I5, I5, I3	3 (index 4)	Left	5 12 8 21 3 17 9 11 2 14
6	5, 12, 8, 21, 3	I5, I5, I5, I5, I4	3 (index 4)	Right	5 12 8 21 3 17 9 11 2 14 6

All four went to the next array index in turn, 7, 8, 9 and 10, exactly as I6 says they should.

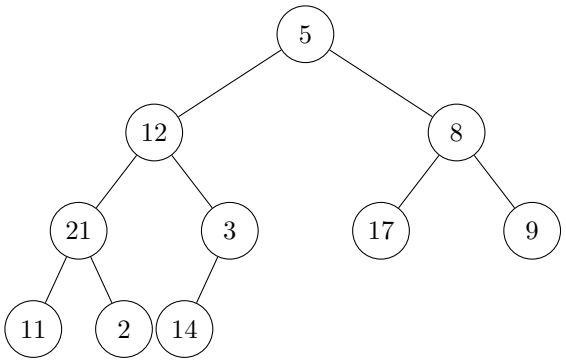
Tree after each insertion



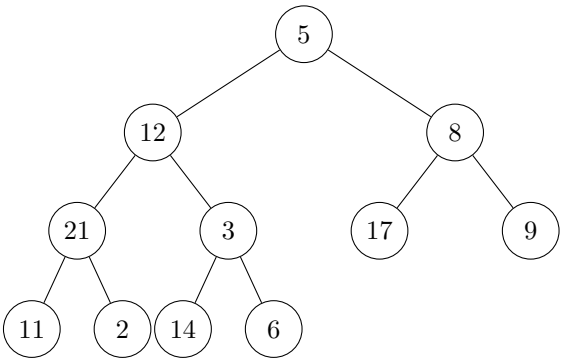
After inserting 11



After inserting 2



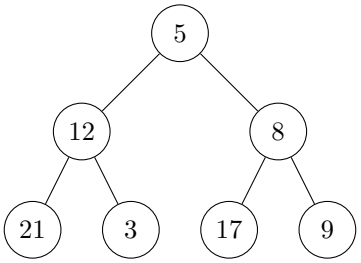
After inserting 14



After inserting 6

PROBLEM 4

Every part starts again from the tree of Problem 1, so the level order before each deletion is always 5 12 8 21 3 17 9. That traversal ends at 9, so by D3 the deepest node is 9 in all of these cases.

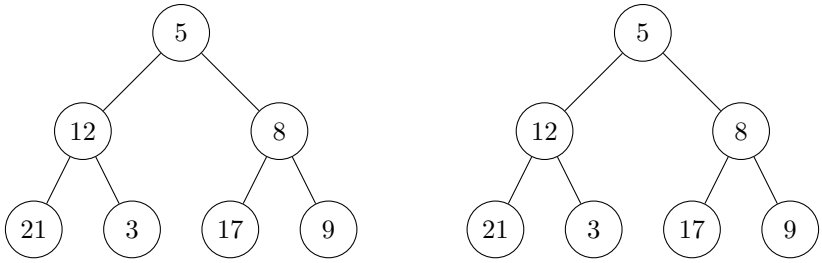


The starting tree for every case.

(a) Case A, delete 4

D3 checks 5, 12, 8, 21, 3, 17 and 9 in level order. None of them is 4, so D4 says the key is not in the tree and the deletion stops there. Nothing is copied and nothing is removed, so the tree does not change.

Rules: D3, D4. Level order after: 5 12 8 21 3 17 9.

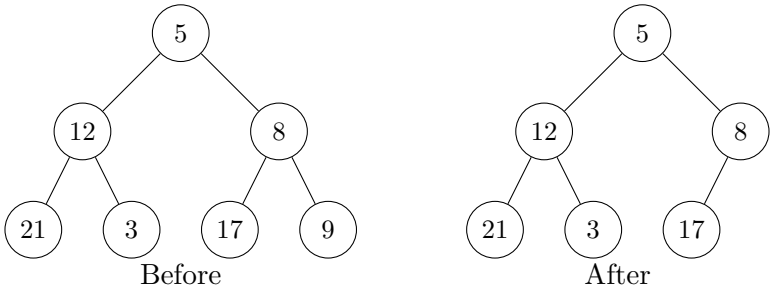


(b) Case B, delete 9

The target and the deepest node are both 9, so this is D7.

This is the easiest case because the copy does no work. D5 would move the deepest value into the target, but they are the same node, so there is nothing to copy. Only D6 runs, and it sets the right pointer of 8 to NULL. No value moves.

Rules: D3, D7. Level order after: 5 12 8 21 3 17.

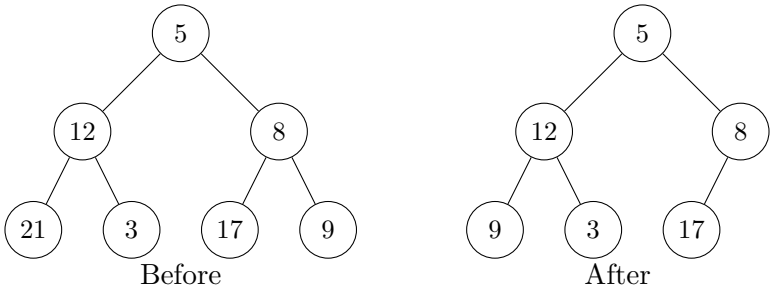


(c) Case C, delete 21

The target 21 is at index 3 and the deepest node 9 is at index 6, so they are different nodes.

D5 copies 9 into the node at index 3. D6 then removes the old 9, so the right pointer of 8 becomes NULL. At index 3 only the data changed, and its own pointers were left alone.

Rules: D3, D5, D6. Level order after: 5 12 8 9 3 17.

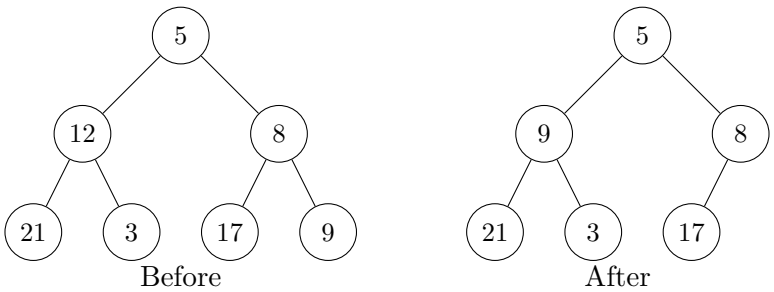


(d) Case D, delete 12

The target 12 is at index 1 and has two children. D5 copies 9 into it and D6 removes the old 9, so the right pointer of 8 becomes NULL.

The replacement node ends up with the children 21 and 3, the same two 12 had. D5 only changes the data field, so the pointers are untouched and the children stay attached.

Rules: D3, D5, D6. Level order after: 5 9 8 21 3 17.

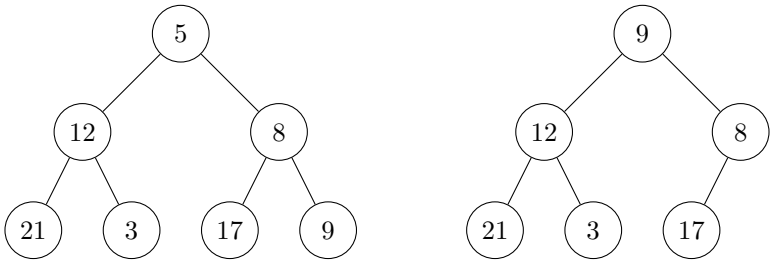


(e) Case E, delete 5

The target 5 is the root at index 0. D5 copies 9 into the root node and D6 removes the old 9.

The new root is 9. The root node itself is not deleted, only its value is overwritten, so both subtrees stay where they were. This is the same as part (d), just at index 0.

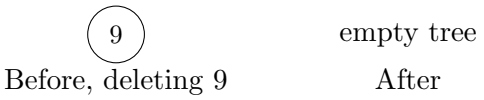
Rules: D3, D5, D6. Level order after: 9 12 8 21 3 17.



(f) Case F, a tree with only one node

The tree is just 9, so $n = 1$ and D2 covers both deletions.

Deleting 9 empties the tree, since the one node holds the key. Deleting 4 on a fresh tree of 9 changes nothing, since the one node does not hold the key.



Deleting 4 instead leaves the single node 9 in place, so the before and after pictures are both the one node above on the left.

(g) Case G, an empty tree

Deleting 5 from an empty tree does nothing, by D1. There is no node to find and none to remove, so the tree stays empty. D1 has to be checked first, or D3 would try to read a root that is not there.



Summary of the deletion cases

Case	Delete	Target	Deepest	Rules applied	Level order after
A	4	not found	9	D3, D4	5 12 8 21 3 17 9, unchanged
B	9	9, the deepest	9	D3, D7	5 12 8 21 3 17
C	21	21, a leaf	9	D3, D5, D6	5 12 8 9 3 17
D	12	12, internal	9	D3, D5, D6	5 9 8 21 3 17
E	5	5, the root	9	D3, D5, D6	9 12 8 21 3 17
F	9 / 4	single node	itself / none	D2	empty / 9, unchanged
G	5	none, empty	none	D1	empty

Checking the drawn trees with the index shortcut

Cases B to E all leave 6 nodes, so the indices go 0 to 5. I will check case C, which is 5 12 8 9 3 17. At index 1 sits 12, and indices 3 and 4 hold 9 and 3, which is what I drew under it. At index 2 sits 8. Index 5 holds 17, so that is its left child. Index 6 does not exist, because $n = 6$, so its right child is NULL. That is what I drew. B, D and E look the same, so they check out the same way.

(h) Why the deepest node is moved into the target's place

If I just removed the target node, the tree would have a gap in the middle. Delete 12 that way and the level order reads 5, empty, 8, 21, 3, 17, 9. The index rules need indices 0 to $n - 1$ to be filled, so none of them would work after that.

The nodes under the target would also be left loose. In part (d), 12 is holding 21 and 3. Remove that node and those two have no parent. D5 avoids this because it only writes over the data, so the pointers and the children stay put.

The deepest node is the only node I can take away for free. It is last in level order, it has no children, and it sits at index $n - 1$. Removing it just shortens the filled range by one. So D5 clears the value and D6 does the removal at the safe spot, and the tree stays complete.

PROBLEM 5

There is no reset here. Each deletion works on what the last one left behind, so I have to find the deepest node again every time, and it is a different node at almost every step.

(a) The deletion sequence

#	Delete	Tree before	Deepest	Case	Rules	Tree after
1	5	5 12 8 21 3 17 9	9	E	D3, D5, D6	9 12 8 21 3 17
2	12	9 12 8 21 3 17	17	D	D3, D5, D6	9 17 8 21 3
3	8	9 17 8 21 3	3	C	D3, D5, D6	9 17 3 21
4	21	9 17 3 21	21	B	D3, D7	9 17 3
5	3	9 17 3	3	B	D3, D7	9 17
6	17	9 17	17	B	D3, D7	9
7	9	9	9	F	D2	empty

Step 1. The target 5 is the root and the deepest node is 9. D5 copies 9 into the root. D6 removes the old 9, so 8's right pointer becomes NULL.

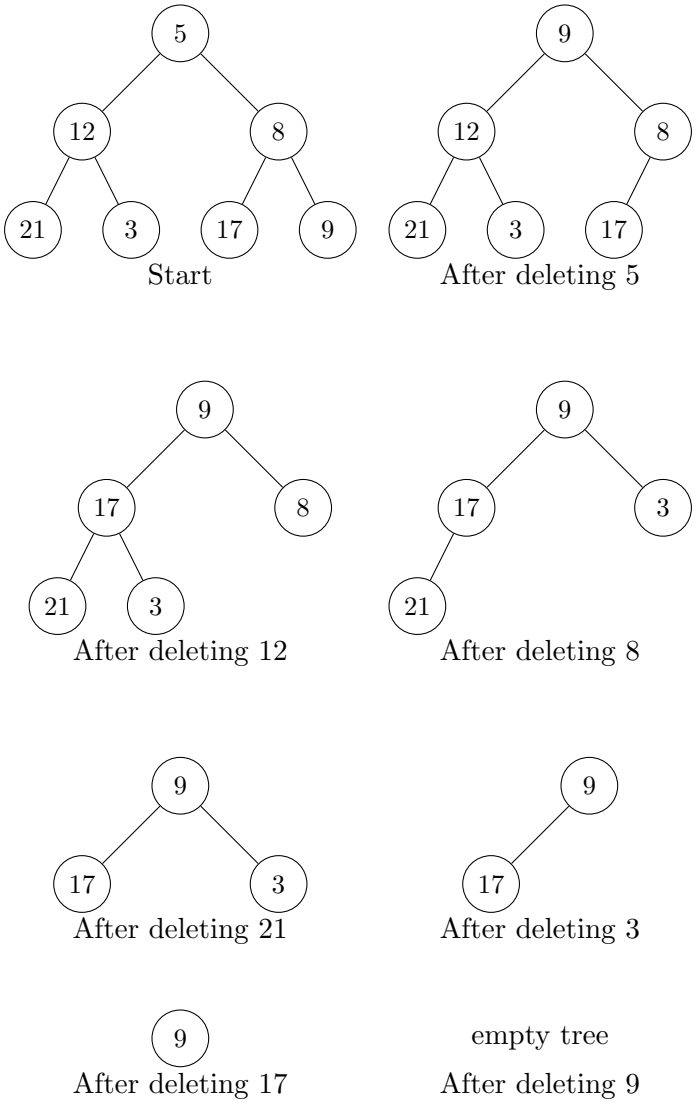
Step 2. Now 12 is internal and the deepest node is 17. D5 copies 17 into 12's node. D6 removes the old 17, so 8's left pointer becomes NULL. That leaves 8 with no children.

Step 3. 8 had two children at the start, but step 1 took 9 and step 2 took 17, so 8 is a leaf now. It is still not the deepest node, because 21 and 3 are a level below it. That makes it case C and not case B. The deepest node is 3. D5 copies 3 into 8's node and D6 removes the old 3, so 17's right pointer becomes NULL.

Steps 4, 5 and 6. Each target is the deepest node now, so D7 applies and nothing gets copied. 21 comes off 17's left, then 3 comes off 9's right, then 17 comes off 9's left.

Step 7. Only 9 is left and it holds the key, so D2 empties the tree.

(b) The tree after every deletion



(c) Which cases came up, and was the tree always complete

The cases used, in order, were E, D, C, B, B, B and F.

Case	Where it came up	How many times
E, the root	step 1	1
D, an internal node	step 2	1
C, a leaf not deepest	step 3	1
B, target is deepest	steps 4, 5, 6	3
F, a single node	step 7	1
A, key not found	never	0
G, an empty tree	never	0

A and G did not come up. Every key I deleted was in the tree, so D4 was never needed. No step started on an empty tree, so D1 was never needed. The three harder cases all come from deleting 5, 12 and 8 first. After that only case B is left.

The tree was complete after every step. The sizes go 7, 6, 5, 4, 3, 2, 1, 0. At each size the nodes fill indices 0 up to $n - 1$ and nothing is missing.

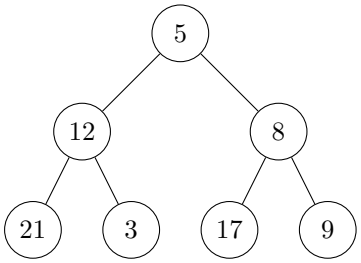
Only one node is ever really removed, and that is always the deepest one. It sits at index $n - 1$. Taking it off makes the filled range one shorter, so no gap opens up. The deleted value does not leave a hole either, because D5 writes over it where it is.

At the end the order I am deleting in matches level order read backwards. From there the target and the deepest node are always the same node, which is why case B repeats.

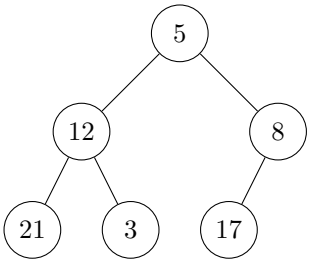
BONUS

Delete order: 9, 17, 3, 21, 8, 12, 5. No reset between deletions.

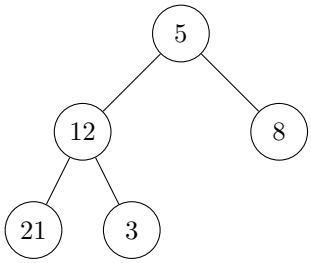
#	Delete	Tree before	Deepest	Case	Rules	Copies	Tree after
1	9	5 12 8 21 3 17 9	9	B	D3, D7	0	5 12 8 21 3 17
2	17	5 12 8 21 3 17	17	B	D3, D7	0	5 12 8 21 3
3	3	5 12 8 21 3	3	B	D3, D7	0	5 12 8 21
4	21	5 12 8 21	21	B	D3, D7	0	5 12 8
5	8	5 12 8	8	B	D3, D7	0	5 12
6	12	5 12	12	B	D3, D7	0	5
7	5	5	5	F	D2	0	empty



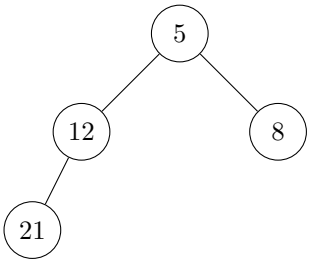
Start



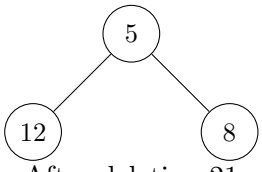
After deleting 9



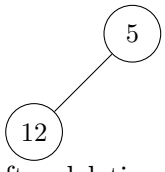
After deleting 17



After deleting 3



After deleting 21



After deleting 8



After deleting 12

empty tree

After deleting 5

Answers

- Case at each step: B for steps 1 to 6, F for step 7.
- Why: 9, 17, 3, 21, 8, 12, 5 is level order reversed. The deepest node is the last in level order, so the target is always the deepest node and D7 applies.
- Why it is simplest: D7 skips the copy. Each deletion is one pointer set to NULL.
- Copies by D5: forward order 3, at steps 1, 2 and 3. Reverse order 0.
- The tree stayed complete at every step, going 7, 6, 5, 4, 3, 2, 1, 0 nodes, since the node removed was always the last one in level order.